# Reviewer Pack — Minimal Number of Abelian Variants in Boolean Satisfiability

Entry d76f24ab682d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 00:34:07 UTC

## Minimal Number of Abelian Variants in Boolean Satisfiability

**Entry ID**: `d76f24ab682d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 00:34:07 UTC

### 1. Conjecture

**Field A** (mathematical branch): Abelian Group Theory **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

**Statement**:

> For every Boolean satisfiability instance φ with n variables, the minimal number of abelian variants required to reduce φ's resolution proof width is bounded by a polynomial in n, specifically |A(φ)| ≤ Θ(n^2), where A(φ) denotes the set of all abelian group structures associated with φ.

**Rationale (proposer's reasoning)**:

> Abelian group theory provides a structured framework for understanding symmetries and reductions. The minimal number of abelian variants could reveal hidden symmetries in Boolean satisfiability instances, leading to more efficient resolution proofs. This connection has not been explored before, as the application of group theory to complexity measures is rare.

**Taxonomy category**: `AbelianGroupTheoryBooleanSatisfiability` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `57ea1cd444fc692f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if, across all 30 random seeds and instances with up to 40 variables, the average minimal number of abelian variants (A(φ)) required to reduce resolution proof width is less than or equal to n^2 * 1.5.

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | UNCERTAIN | 1.00 | HITS | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

### 5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n):
        return [random.choice([True, False]) for _ in range(n)]

    def construct_abelian_variants(instance):
        n = len(instance)
        abelian_variants = set()
        for i in range(1 << n):
            variant = []
            for j in range(n):
                if (i >> j) & 1:
                    variant.append(instance[j])
            abelian_variants.add(tuple(variant))
        return abelian_variants

    def measure_resolution_width(instance):
        # Placeholder for actual resolution width calculation
        # This is a dummy implementation for testing purposes
        return len(instance)

    n_max = 0
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        instance = generate_instance(n)
        abelian_variants = construct_abelian_variants(instance)
        resolution_width = measure_resolution_width(instance)
        metric_value = len(abelian_variants) / (n ** 2 * 1.5)
```

```
        total_metric_value += metric_value
        instances_tested += n
        n_max = max(n_max, n)

        if conjecture_holds and metric_value > 1:
            conjecture_holds = False
            counterexample = f"Instance size {n} with resolution width {resolution_width}"

    return {
        "metric_name": "abelian_variants",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test timed out before producing data, which means it did not complete within the allotted time frame. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time constraints.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 15163 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 9678 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 11298 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8635 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 14304 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 9402 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 18614 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 9746 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 18158 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 114999 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/d76f24ab682d.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d76f24ab682d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/d76f24ab682d.tar.gz` (if generated)